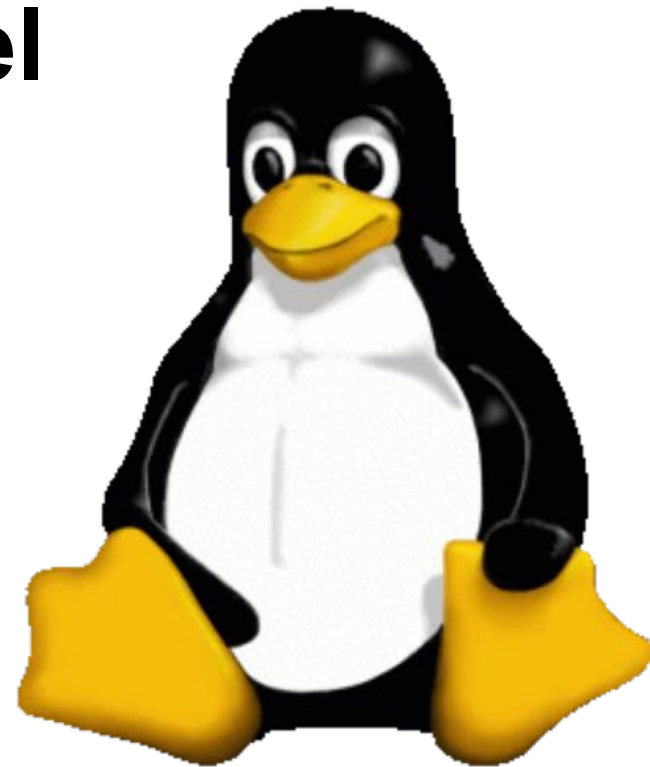


CS353 Linux Kernel

Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn

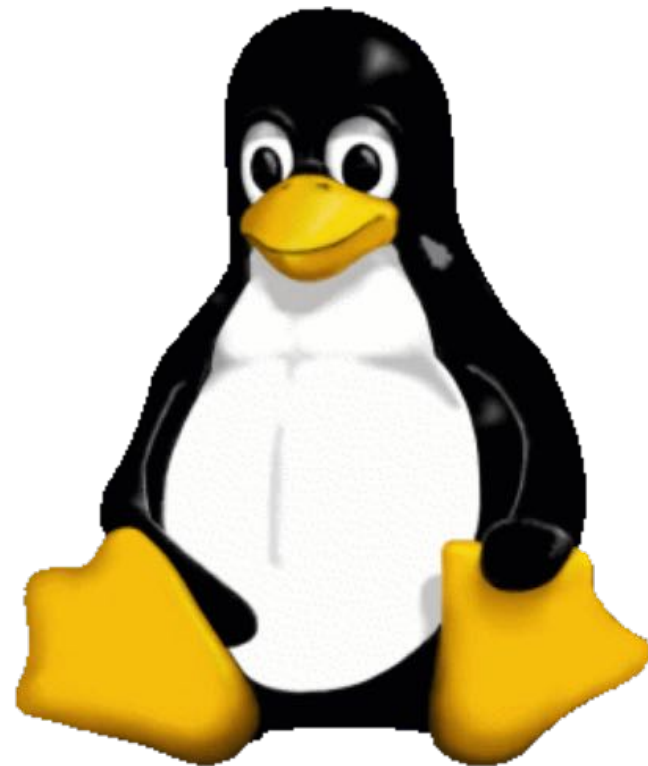


上海交通大学



7A. Virtual File System

Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



上海交通大学

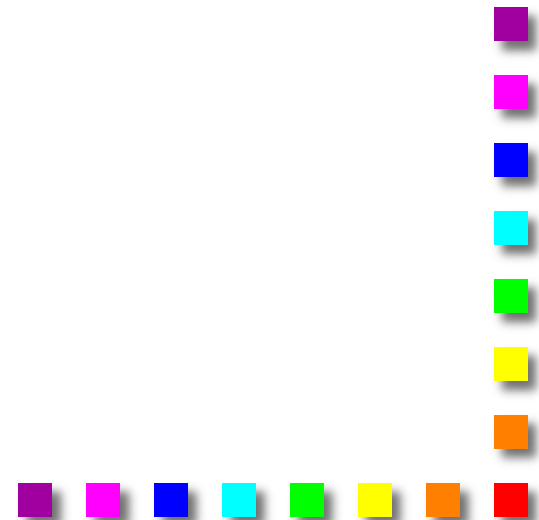


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking



上海交通大学

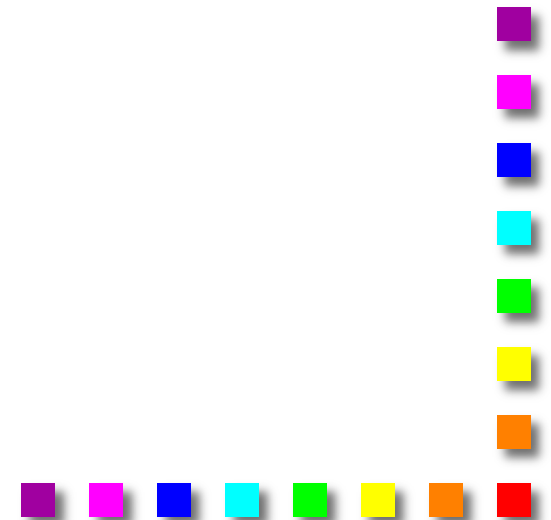


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking

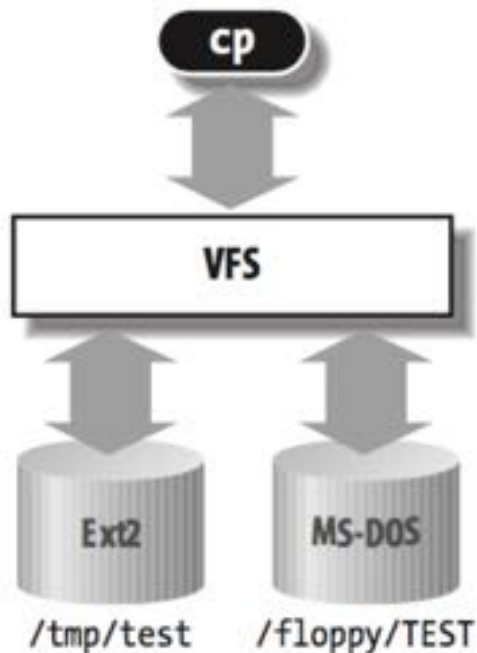


上海交通大学



Role of VFS (1)

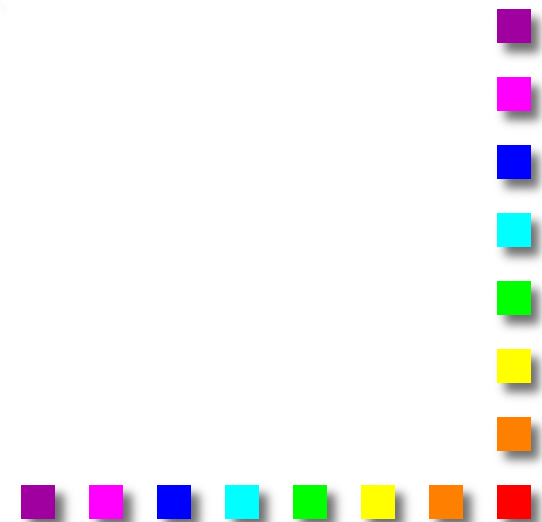
- A common interface to several kinds of filesystems
 - Ex: `cp /floppy/TEST /tmp/test`



(a)

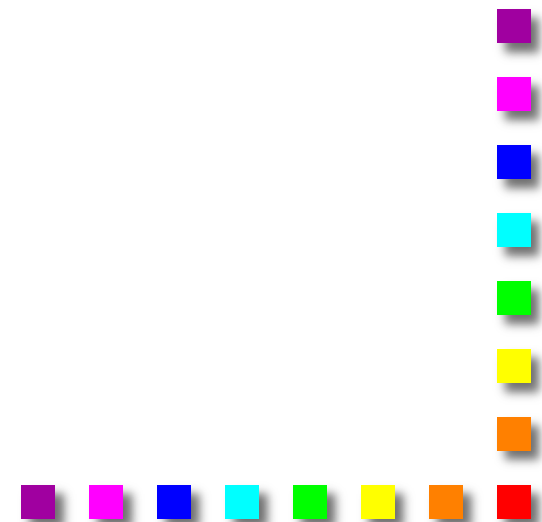
```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
            O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

(b)



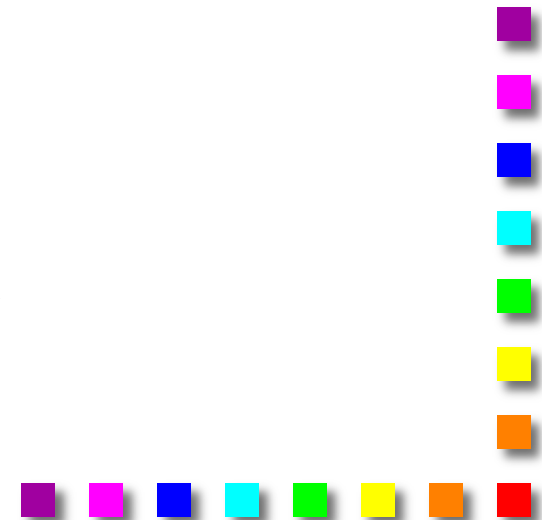
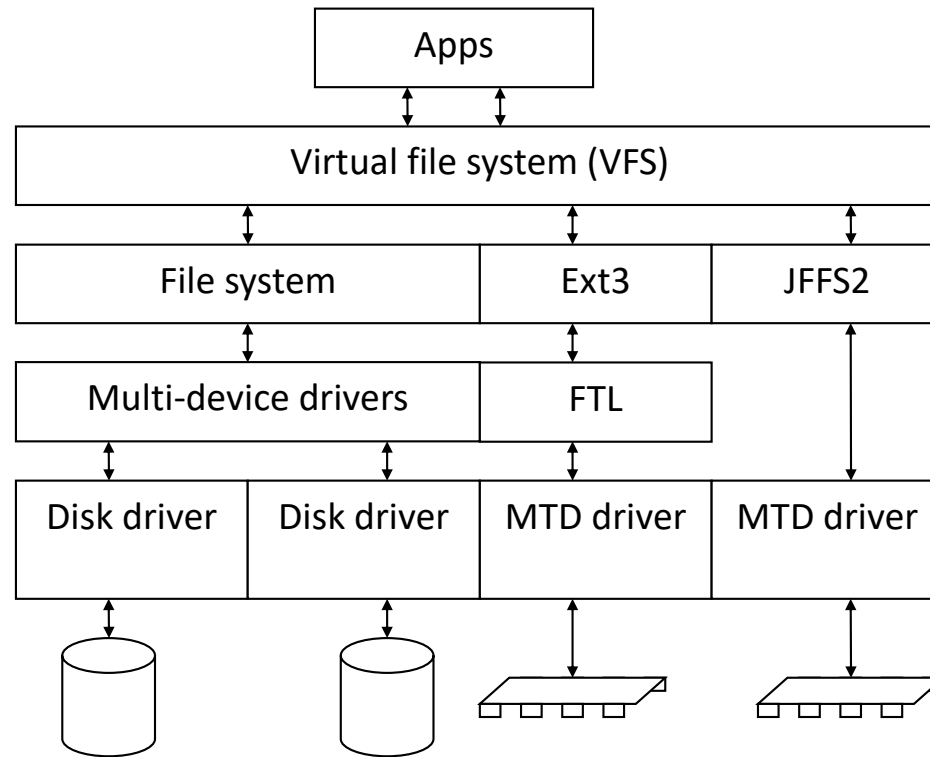
Role of VFS (2)

- Filesystems supported by the VFS
 - Disk-based filesystems
 - Ext2, ext3, ReiserFS
 - Sysv, UFS, MINIX, VxFS
 - VFAT, NTFS
 - ISO9660 CD-ROM, UDF DVD
 - HPFS, HFS, AFFS, ADFS,
 - Network filesystems
 - NFS, Coda, AFS, CIFS, NCP
 - Special filesystems
 - E.g. /proc



Common File System Interface

- Enables system calls such as open(), read(), and write() to work regardless of file system or storage media

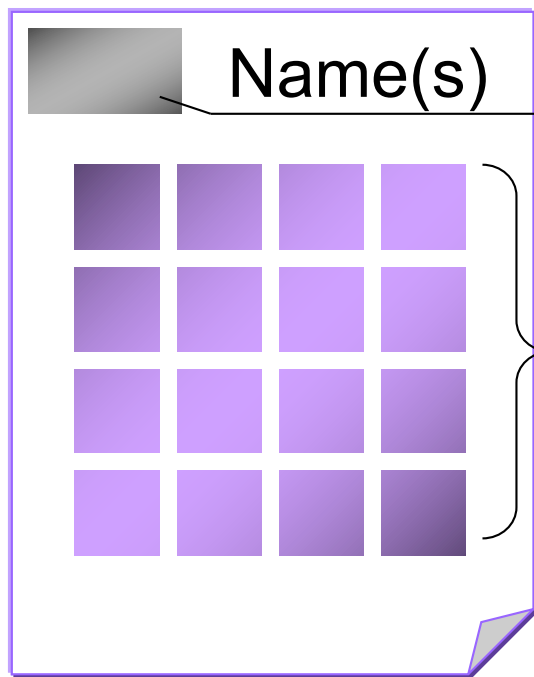


Terminology

- **File system** – storage of data adhering to a specific structure
- **File** – ordered string of bytes
- **Directory** – analogous to a folder
 - Special type of file
 - Instead of normal data, it contains “pointers” to other files
 - Directories are hooked together to create the hierarchical namespace
- **Metadata** – information describing a file



Physical File Representation



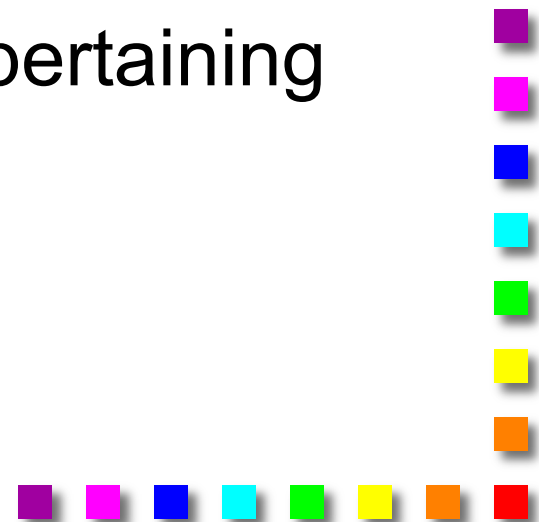
File

Inode

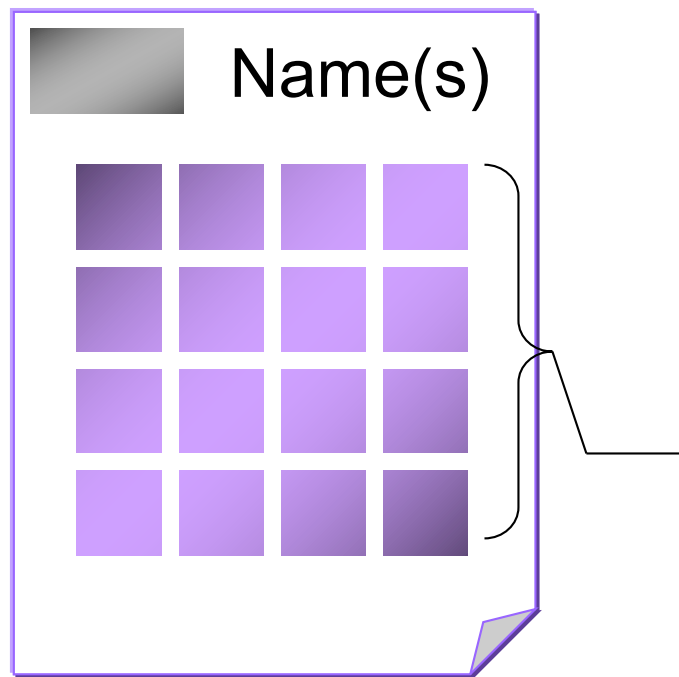
- Unique index
- Holds file attributes and data block locations pertaining to a file



上海交通大学

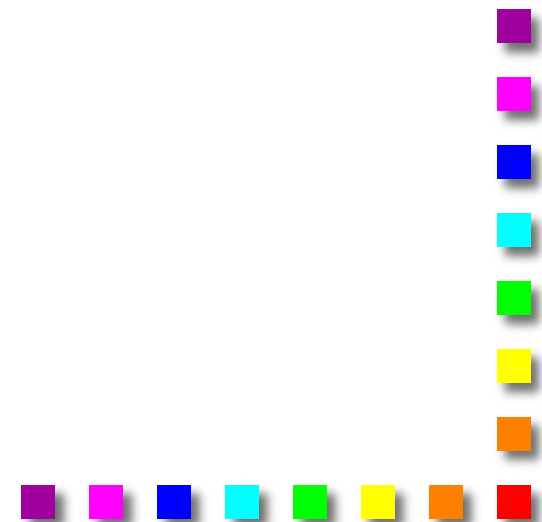


Physical File Representation

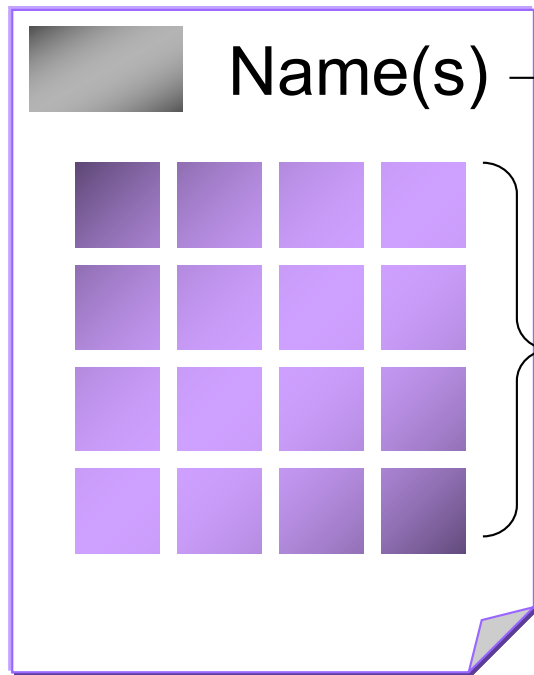


File

- *Data blocks*
 - Contains file data
 - May not be physically contiguous



Physical File Representation

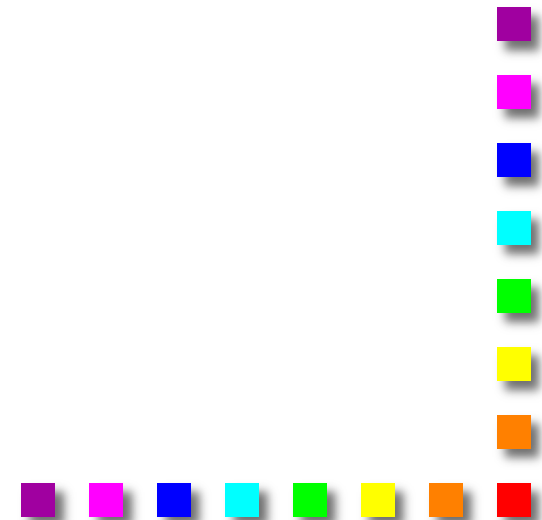


File

- *File name*
 - Human-readable identifier for each file

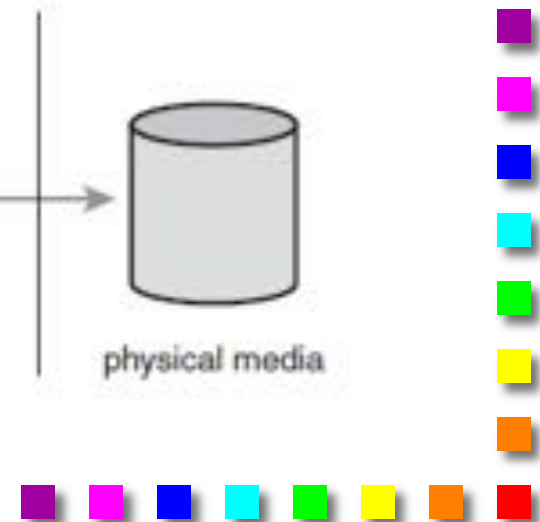
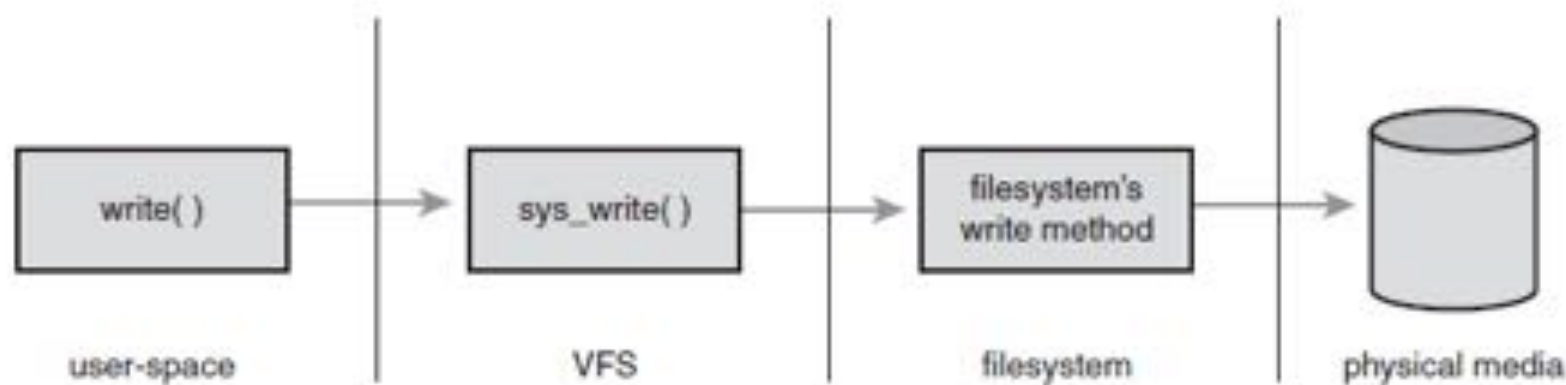


上海交通大学



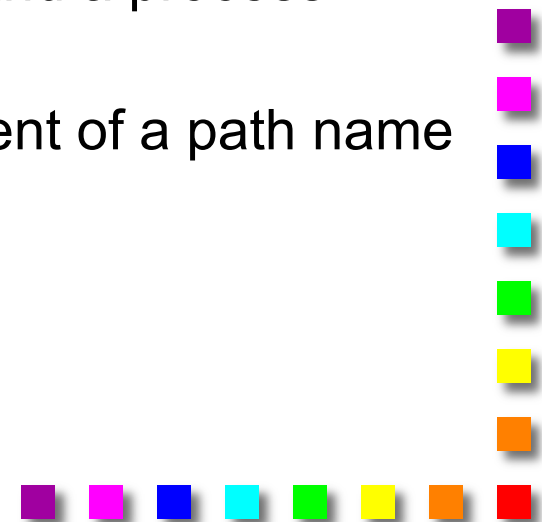
The Common File Model (1)

- Defines common file model conceptual interfaces and data structures
- Capable of representing all supported filesystems
 - Each specific filesystem implementation must translate its physical organization into VFS's common file model
 - E.g.: `write(...): file->f_op->write(...);`

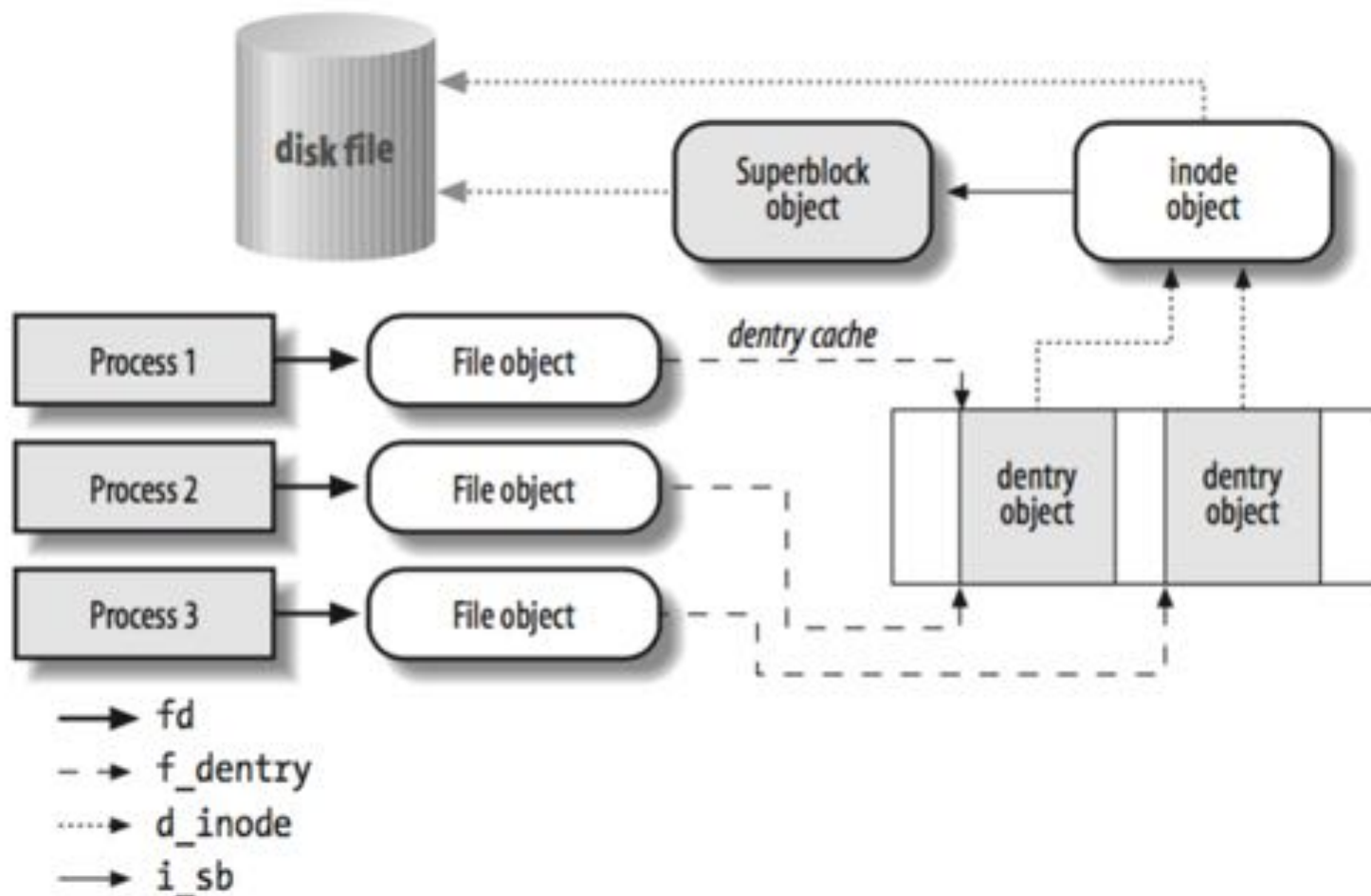


The Common File Model (2)

- Object-oriented: data structures and associated operations
 - Superblock object: a mounted filesystem
 - Inode object: information about a file, represents a specific file
 - File object: interaction between an open file and a process
 - Dentry object: directory entry, single component of a path name

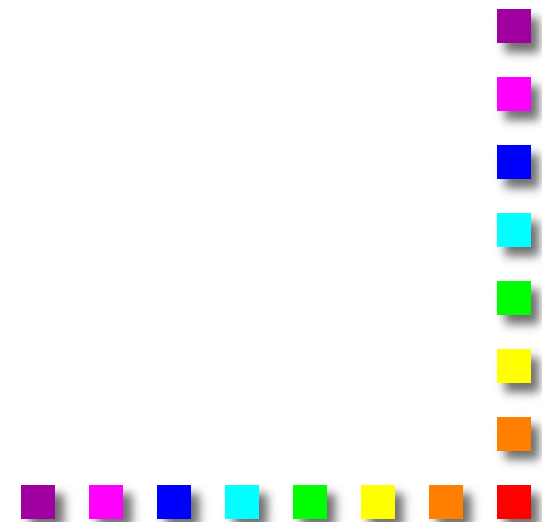


The Common File Model (2)



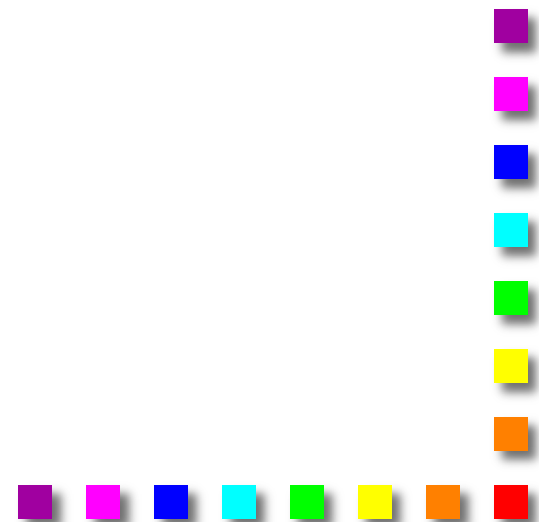
VFS Operations

- Each object contains operations object with methods
 - super_operations -- invoked on a specific file system
 - inode_operations -- invoked on a specific inodes (which point to a file)
 - dentry_operations -- invoked on a specific directory entry
 - file_operations -- invoked on a file



VFS Operations

- Lower file system can implement own version of methods to be called by VFS
- If an operation is not defined by a lower file system (NULL), VFS will often call a generic version of the method
- Example shown on next slide...



VFS Operations

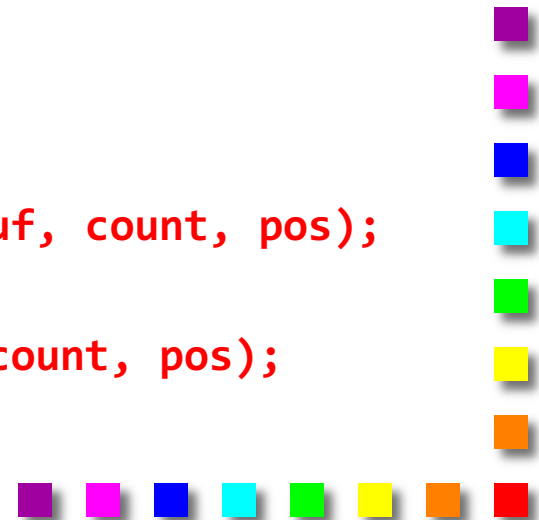
```
ssize_t vfs_write(struct file *file, const char __user *buf, size_t
count, loff_t *pos)
{
    ssize_t ret;

    /* Misc file checks (snip) ... */

    ret = rw_verify_area(WRITE, file, pos, count);
    if (ret >= 0) {
        count = ret;
        if (file->f_op->write)
            ret = file->f_op->write(file, buf, count, pos);
        else
            ret = do_sync_write(file, buf, count, pos);
    }
}
```



上海交通大学



Some System Calls Handled by the VFS (1)

■ Filesystem

- Mount(), umount(), umount2()
- Sysfs()
- Statfs(), fstatfs(), statfs64(), fstatfs64(), ustat()

■ Directories

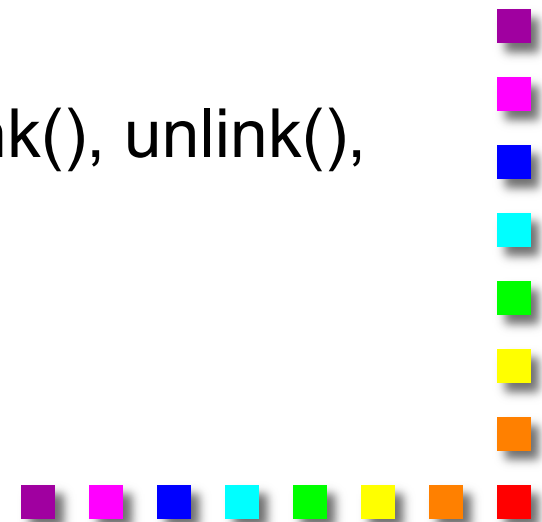
- Chroot(), pivot_root()
- Chdir(), fchdir(), getcwd()
- Mkdir(), rmdir()
- Getdents(), getdents64(), readdir(), link(), unlink(), rename(), lookup_dcookie()

■ Links

- Readlink(), symlink()



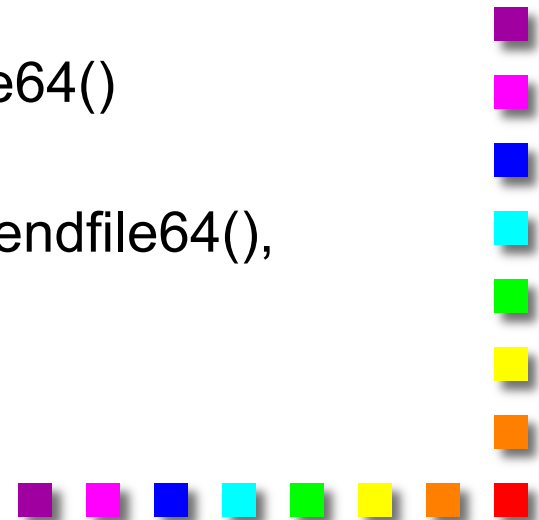
上海交通大学



Some System Calls Handled by the VFS (2)

■ Files

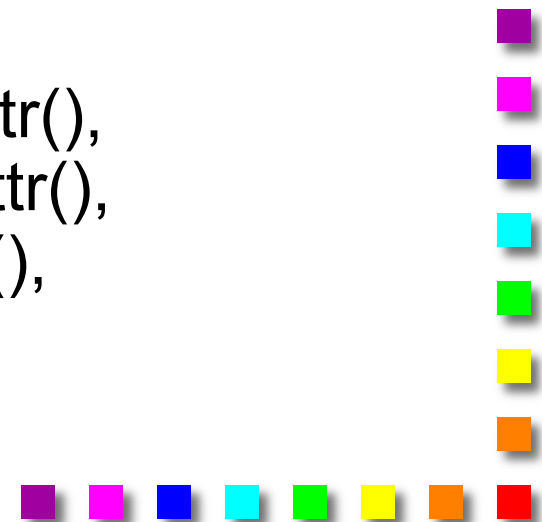
- Chown(), fchown(), lchown(), chown16(), fchown16(), lchown16()
- Chmod(), fchmod(), utime()
- Stat(), fstat(), lstat(), access(), oldstat(), oldfstat(), oldlstat(), stat64(), lstat64(), fstat64()
- Open(), close(), creat(), umask()
- Dup(), dup2(), fcntl(), fcntl64()
- Select(), poll()
- Truncate(), ftruncate(), truncate64(), ftruncate64()
- Lseek(). _llseek()
- Read(), write(), readv(), writev(), sendfile(), sendfile64(), readahead()



Some System Calls Handled by the VFS (3)

■ Others

- `io_setup()`, `io_submit()`, `io_getevents()`,
`io_cancel()`, `io_destroy()`
- `Pread64()`, `pwrite64()`
- `Mmap()`, `mmap2()`, `munmap()`, `madvise()`,
`mincore()`, `remap_file_pages()`
- `Fdatasync()`, `fsync()`, `sync()`, `msync()`
- `Flock()`
- `Setxattr()`, `lsetxattr()`, `fsetxattr()`, `getxattr()`,
`lgetxattr(0)`, `fgetxattr()`, `listxattr()`, `llistxattr()`,
`flistxattr()`, `removexattr()`, `lremovexattr()`,
`fremovexattr()`

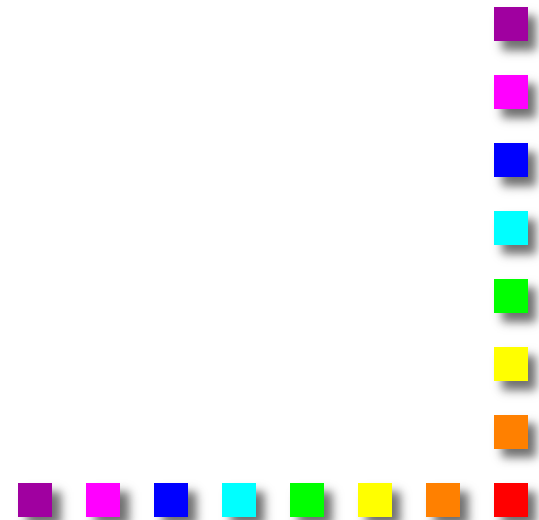


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking

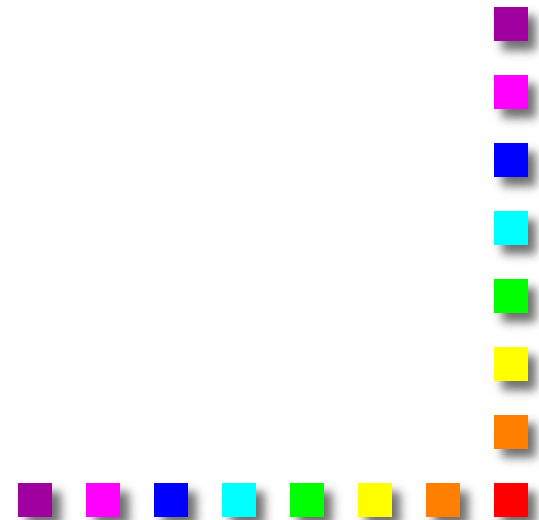


上海交通大学



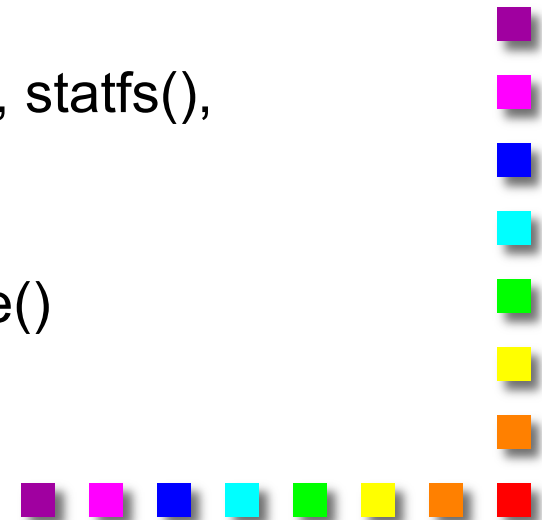
Superblock Object

- Implemented by each file system
- Used to store information describing that specific file system
- Often physically written at the beginning of the partition (filesystem control block)
- Found in `<linux/fs.h>`



VFS Data Structures (1) - Superblock Object

- Superblock objects: super_block structure
 - (Table 12-2)
 - S_op: superblock operations – super_operations structure
 - Alloc_inode(), destroy_inode()
 - Read_inode(), dirty_inode(), write_inode(),
 - Put_inode(), drop_inode(), delete_inode()
 - Put_super(), write_super()
 - Sync_fs(), write_super_lockfs(), unlockfs(), statfs(), remount_fs()
 - Clear_inode(), umount_begin()
 - Show_options(), quota_read(), quota_write()



Superblock Object Struct

```
struct super_block {  
    struct list_head s_list;           /* list of all superblocks */  
    dev_t s_dev;                       /* identifier */  
    unsigned long s_blocksize;         /* block size in bytes*/  
    unsigned char s_blocksize_bits;    /* block size in bits*/  
    unsigned char s_dirt;               /* dirty flag */  
    unsigned long long s_maxbytes;     /* max file size */  
    struct file_system_type s_type;    /* filesystem type */  
    struct super_operations s_op;      /* superblock methods*/  
    struct dquot_operations *dq_op;    /* quota methods */  
    struct quotactl_ops *s_qcop;       /* quota control */  
    struct export_operations *s_export_op; /* export methods */  
    unsigned long s_flags;             /* mount flags */  
    unsigned long s_magic;             /* FS magic number */  
    struct dentry *s_root;             /* dir mount point*/  
};
```

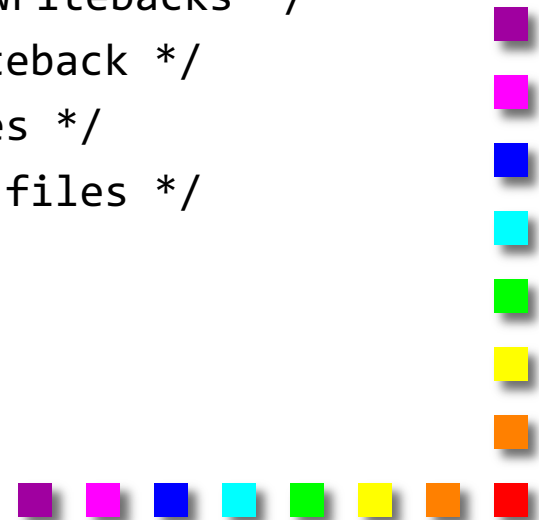


上海交通大学



Superblock Object Struct (cont.)

```
struct rw_semaphore s_umount;      /* unmount semaphore */
struct semaphore s_lock;           /* superblock semaphore */
int s_count;                       /* superblock ref count */
int s_need_sync;                   /* not-yet-synced flag */
atomic_t s_active;                 /* active reference count */
void *s_security;                  /* security module */
struct xattr_handler **s_xattr;    /* extended attribute handlers */
struct list_head s_inodes;         /* list of inodes */
struct list_head s_dirty;         /* list of dirty inodes */
struct list_head s_io;             /* list of writebacks */
struct list_head s_more_io;        /* list of more writeback */
struct hlist_head s_anon;          /* anonymous dentries */
struct list_head s_files;          /* list of assigned files */
```



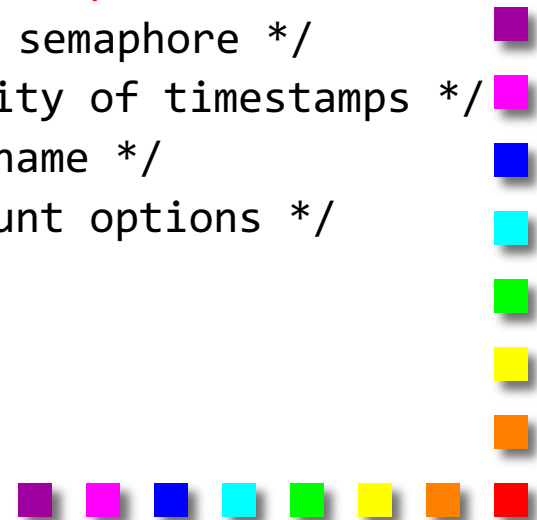
Superblock Object Struct (cont.)

```
struct list_head s_dentry_lru; /* list of unused dentries */
int s_nr_dentry_unused; /* number of dentries on list*/
struct block_device *s_bdev; /* associated block device */
struct list_head s_instances; /* instances of this fs */
struct quota_info s_dquot; /* quota-specific options */
int s_frozen; /* frozen status */
wait_queue_head_t s_wait_unfrozen; /* wait queue on freeze */
char s_id[32]; /* text name */
void *s_fs_info; /* filesystem-specific info */
struct semaphore s_vfs_rename_sem; /* rename semaphore */
u32 s_time_gran; /* granularity of timestamps */
char *s_subtype; /* subtype name */
char *s_options; /* saved mount options */
```

}

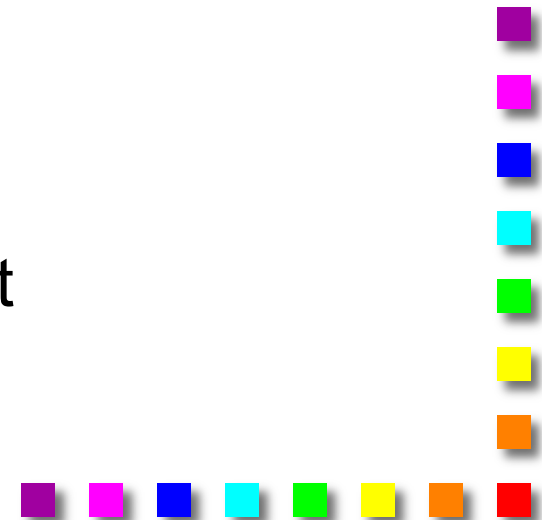


上海交通大学



Inode Object

- Represents all the information needed to manipulate a file or directory
- Constructed in memory, regardless of how file system stores metadata information
- Three doubly linked lists
 - List of valid unused inodes
 - List of in-use inodes
 - List of dirty inodes
- Per-filesystem doubly linked circular list
- A hash table named `inode_hashtable`



VFS Data Structures (2) - Inode

- Inode objects: inode structure
 - i_op: inode operations – inode_operations structure
 - Create(), lookup(), link(), unlink(), symlink()
 - Mkdir(), rmdir(), mknod(), rename()
 - readlink(), follow_link(), put_link()
 - Truncate(), permission(),
 - Setattr(), getattr(), setxattr(), getxattr(), listxattr(), removexattr()

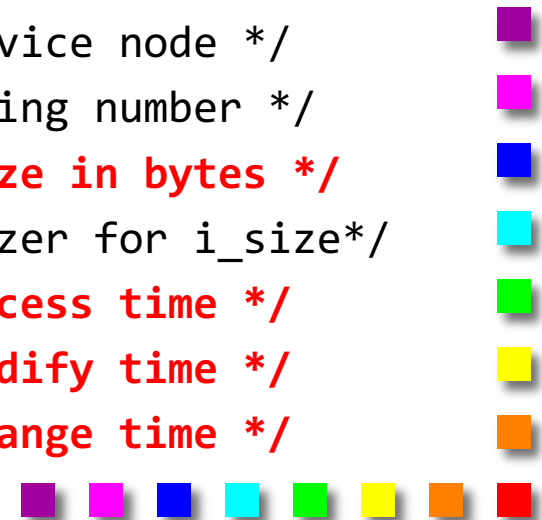


Inode Object Struct

```
struct inode {  
    struct hlist_node i_hash;           /* hash list */  
    struct list_head i_list;           /* list of inodes */  
    struct list_head i_sb_list;        /* list of superblocks */  
    struct list_head i_dentry;         /* list of dentries */  
    unsigned long i_ino;                /* inode number */  
    atomic_t i_count;                  /* reference counter */  
    unsigned int i_nlink;               /* number of hard links */  
    uid_t i_uid;                       /* user id of owner */  
    gid_t i_gid;                       /* group id of owner */  
    kdev_t i_rdev;                     /* real device node */  
    u64 i_version;                     /* versioning number */  
    loff_t i_size;                     /* file size in bytes */  
    seqcount_t i_size_seqcount;        /* serializer for i_size*/  
    struct timespec i_atime;           /* last access time */  
    struct timespec i_mtime;           /* last modify time */  
    struct timespec i_ctime;           /* last change time */  
};
```



上海交通大学



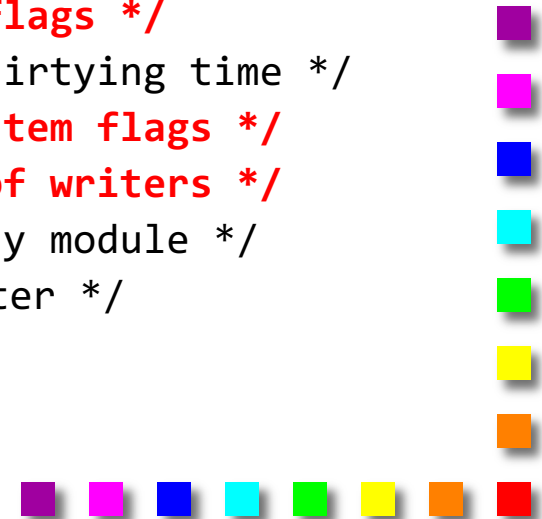
Inode Object Struct (cont.)

```
unsigned int i_blkbits;           /* block size in bits */
blkcnt_t i_blocks;               /* file size in blocks */
unsigned short i_bytes;          /* bytes consumed */
spinlock_t i_lock;             /* spinlock */
struct rw_semaphore i_alloc_sem; /* nests inside of i_sem */
struct semaphore i_sem;        /* inode semaphore */
struct inode_operations *i_op;   /* inode ops table */
struct file_operations *i_fop; /* default inode ops */
struct super_block *i_sb;      /* associated superblock */
struct file_lock *i_flock;       /* file lock list */
struct address_space *i_mapping; /* associated mapping */
struct address_space i_data;      /* mapping for device */
struct dquot *i_dquot[MAXQUOTAS]; /* disk quotas for inode */
struct list_head i_devices;       /* list of block devices */
```



Inode Object Struct (cont.)

```
union {
    struct pipe_inode_info *i_pipe; /* pipe information */
    struct block_device *i_bdev; /* block device driver */
    struct cdev *i_cdev; /* character device driver */
};
unsigned long i_dnotify_mask; /* directory notify mask */
struct dnotify_struct *i_dnotify; /* dnotify */
struct list_head inotify_watches; /* inotify watches */
struct mutex inotify_mutex; /* protects inotify_watches */
unsigned long i_state; /* state flags */
unsigned long dirtied_when; /* first dirtying time */
unsigned int i_flags; /* filesystem flags */
atomic_t i_writecount; /* count of writers */
void *i_security; /* security module */
void *i_private; /* fs private pointer */
};
```



Dentry Object

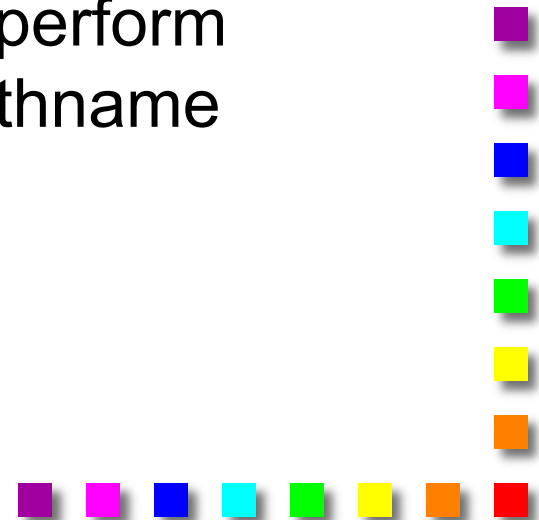
- VFS treats directories as a type of file
- Example

`/bin/vi`

- Both `bin` and `vi` are files
- Each file has an inode representation
- However, sometimes VFS needs to perform directory-specific operations, like pathname lookup



上海交通大学



Dentry Object

- Dentry (directory entry) is a specific component in a path
 - Dentry objects:
 - `"/`
 - `"bin"`
 - `"vi"`
- Represented by `struct dentry` and defined in `<linux/dcache.h>`



VFS Data Structures (3) – Dentry

- Dentry objects: (Table 12-5)

- Dentry operations

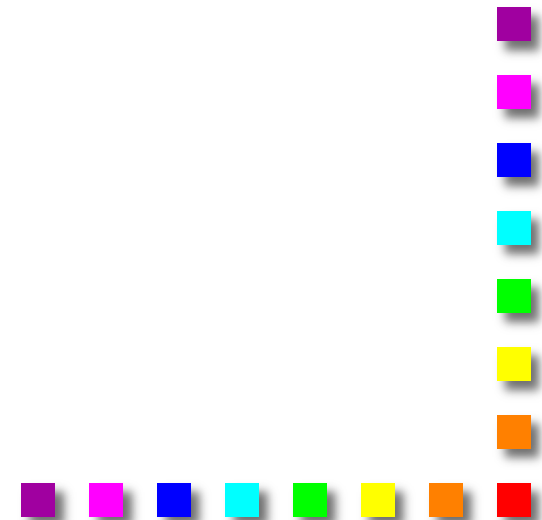
- D_revalidate()
 - D_hash()
 - D_compare()
 - D_delete()
 - D_release()
 - D_input()

- Dentry cache

- A set of dentry objects
 - A hash table



上海交通大学



Dentry Object Struct

```
struct dentry {  
    atomic_t d_count;           /* usage count */  
    unsigned int d_flags;       /* dentry flags */  
    spinlock_t d_lock;         /* per-dentry lock */  
    int d_mounted;             /* is this a mount point? */  
    struct inode *d_inode;      /* associated inode */  
    struct hlist_node d_hash;   /* list of hash table entries*/  
    struct dentry *d_parent;    /* dentry object of parent */  
    struct qstr d_name;         /* dentry name */  
    struct list_head d_lru;     /* unused list */  
    union {  
        struct list_head d_child; /* list of dentries within */  
        struct rcu_head d_rcu;    /* RCU locking */  
    } d_u;  
};
```

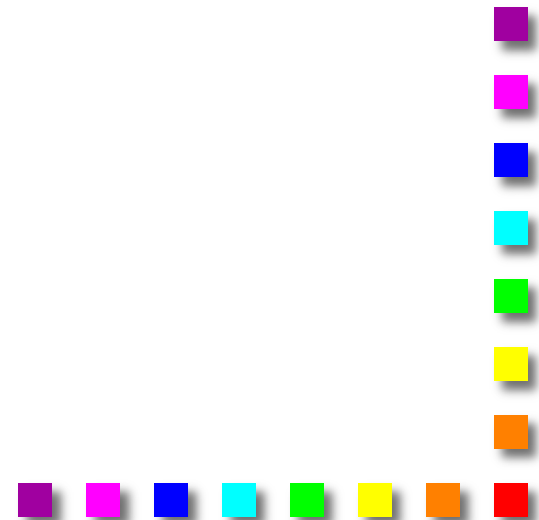


Dentry Object Struct (cont.)

```
struct list_head d_subdirs; /* subdirectories */
struct list_head d_alias;   /* list of alias inodes */
unsigned long d_time;       /* revalidate time */
struct dentry_operations *d_op; /* dentry operations table */
struct super_block *d_sb;    /* superblock of file */
void *d_fsdata;             /* filesystem-specific data */
unsigned char d_iname[DNAME_INLINE_LEN_MIN]; /* short name */
...
};
```

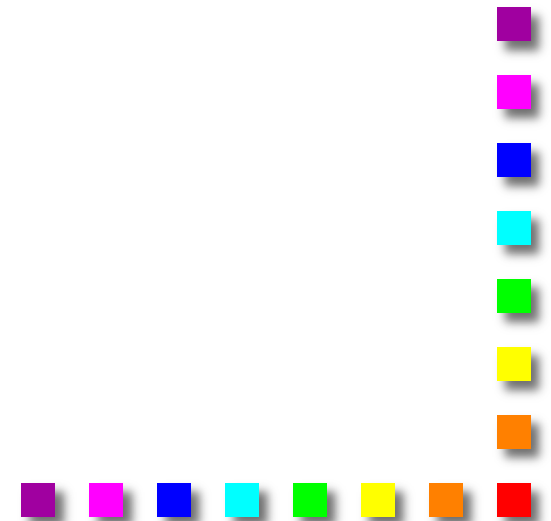


上海交通大学



Dentry State

- Dentry object can be in one of 4 states:
 - Free
 - Unused
 - In use
 - Negative



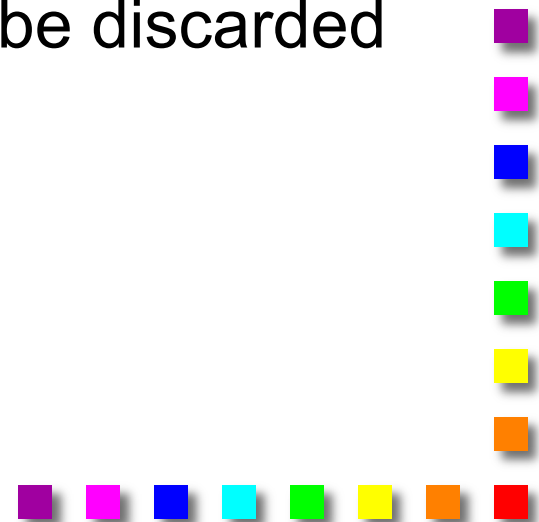
Dentry State

- Unused dentry state
 - Corresponds to a valid inode
 - `d_inode` points to an associated inode
 - **Zero** users of the object
 - `d_count` is zero
 - Since dentry points to valid object, it is cached
 - Quicker for pathname lookups
 - Can be discarded if necessary to reclaim more memory



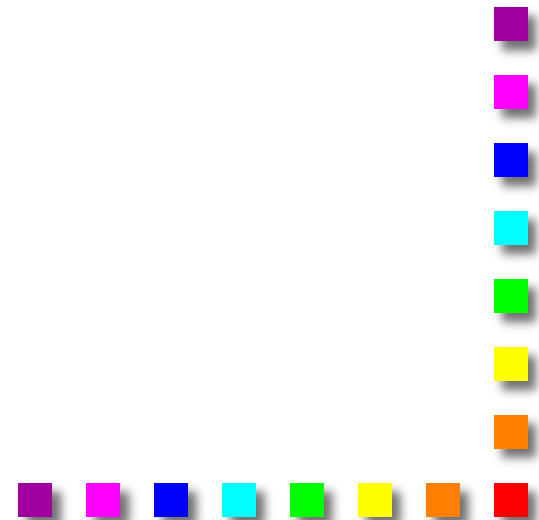
Dentry State

- In use dentry state
 - Corresponds to a valid inode
 - **d_inode** points to an associated inode
 - One or more users of the object
 - **d_count** is positive
 - Dentry is in use by VFS and cannot be discarded



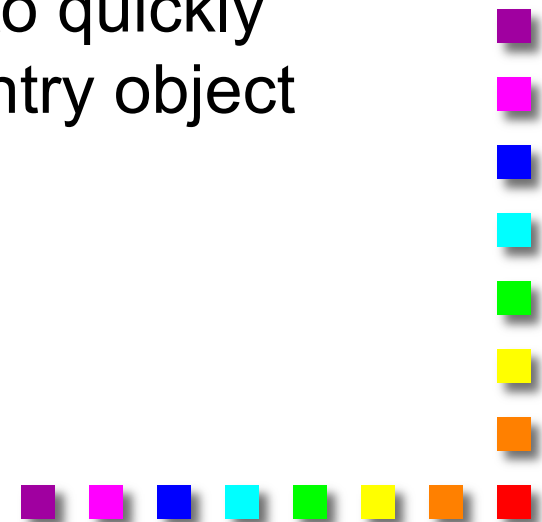
Dentry State

- Negative dentry state
 - Not associated to a valid inode
 - `d_inode` points to NULL
 - Two reasons
 - Program tries to open a file that does not exist
 - Inode of the file was deleted
 - May be cached



Dentry Cache

- Dentry objects stored in a dentry cache
- Cache consists of three parts
 - Lists of used dentries
 - Doubly linked “least recently used” list of unused and negative dentry objects
 - Hash table and hash function used to quickly resolve given path to associated dentry object

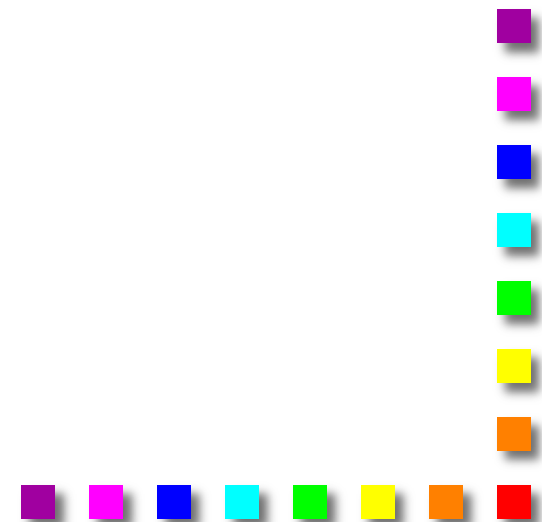


VFS Data Structures (4) -File Object

- Used to represent a file opened by a process
- In-memory representation of an open file
- Represented by **struct file** and defined in **<linux/fs.h>**
- File objects are allocated through a slab cache named *filp*



上海交通大学



File Object Structure

Type	Field	Description
struct list_head	f_list	Pointers for generic file object list
struct dentry *	f_dentry	dentry object associated with the file
struct vfsmount *	f_vfsmnt	Mounted filesystem containing the file
struct file_operations *	f_op	Pointer to file operation table
atomic_t	f_count	File object's reference counter
unsigned int	f_flags	Flags specified when opening the file
mode_t	f_mode	Process access mode
int	f_error	Error code for network write operation
loff_t	f_pos	Current file offset (file pointer)
struct fown_struct	f_owner	Data for I/O event notification via signals
unsigned int	f_uid	User's UID
unsigned int	f_gid	User group ID
struct file_ra_state	f_ra	File read-ahead state (see Chapter 16)
size_t	f_maxcount	Maximum number of bytes that can be read or written with a single operation (currently set to $2^{31}-1$)
unsigned long	f_version	Version number, automatically increased after each use
void *	f_security	Pointer to file object's security structure
void *	private_data	Pointer to data specific for a filesystem or a device driver
struct list_head	f_ep_links	Head of the list of event poll waiters for this file
spinlock_t	f_ep_lock	Spin lock protecting the f_ep_links list
struct address_space *	f_mapping	Pointer to file's address space object (see Chapter 15)



File operations

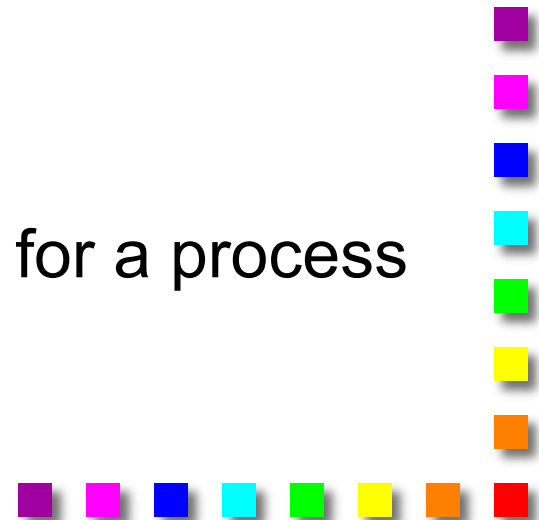
■ File operations

- Lseek(), read(), aio_read(), write(), aio_write()
- Readdir(), poll(), ioctl(), unlocked_ioctl(), compat_ioctl()
- Mmap(), open(), flush(), release()
- Fsync(), aio_fsync(), fasync(), lock()
- Readv(), writev(), sendfile(), sendpage()
- Get_unmapped_area(), check_flags(), dir_notify(), flock()



Files Associated with a Process

- Kernel needs to maintain interactions between a process and a filesystem
- fs field: fs_struct structure
- files field: files_struct structure
 - fd: file descriptors
 - fd[0]: stdin
 - fd[1]: stdout
 - fd[2]: stderr
 - NR_OPEN: max # of file descriptors for a process
 - Usually 1,048,576

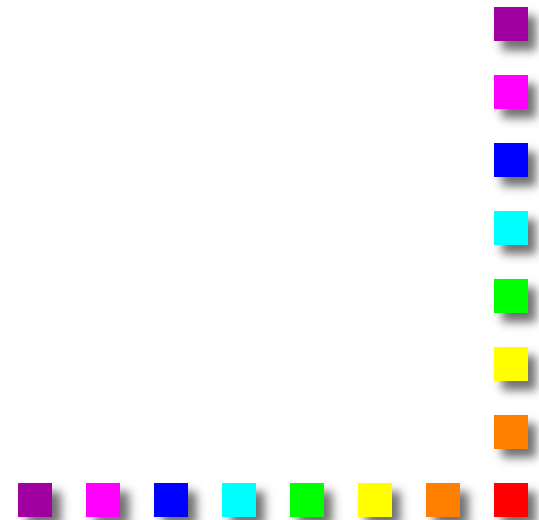


Outline

- Role of VFS
- VFS Data Structures
- **Filesystem Types**
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking



上海交通大学

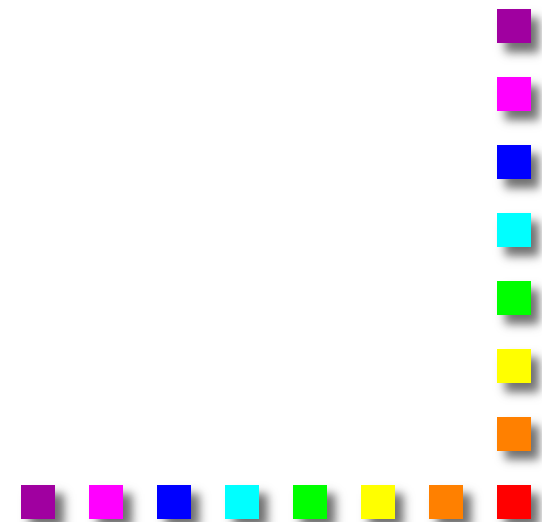


Special Filesystems

- /dev/pts: pseudo terminal support
- /proc: general access point to kernel data structures
- /sys: general access point to system data
- /proc/bus/usb: USB devices
- ...

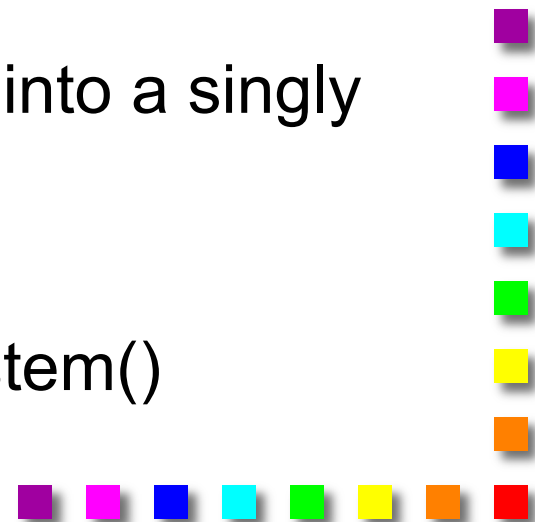


上海交通大学



Filesystem Type Registration (1)

- The VFS keeps track of all filesystem types whose code is currently included in the kernel using *filesystem type registration*
- File_system_type object: a registered file system
- All filesystem-type objects are inserted into a singly linked list
- register_filesystem(), unregister_filesystem()



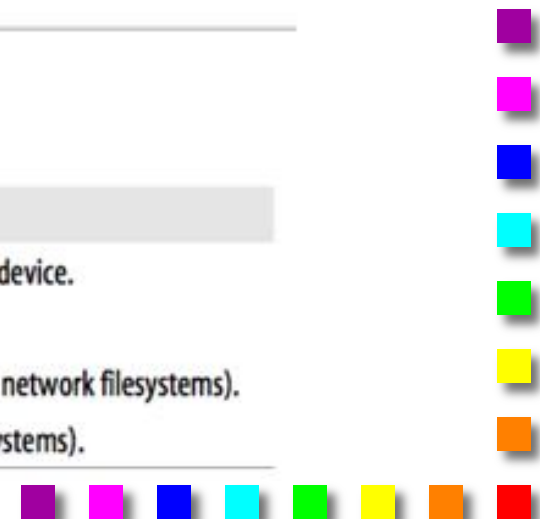
Filesystem Type Registration (2)

■ File_system_type object

Type	Field	Description
const char *	name	Filesystem name
int	fs_flags	Filesystem type flags
struct super_block * (*)()	get_sb	Method for reading a superblock
void (*)()	kill_sb	Method for removing a superblock
struct module *	owner	Pointer to the module implementing the filesystem (see Appendix B)
struct file_system_type *	next	Pointer to the next element in the list of filesystem types
struct list_head	fs_supers	Head of a list of superblock objects having the same filesystem type

■ Fs_flags

Name	Description
FS_REQUIRES_DEV	Every filesystem of this type must be located on a physical disk device.
FS_BINARY_MOUNTDATA	The filesystem uses binary mount data.
FS_REVAL_DOT	Always revalidate the "." and ".." paths in the dentry cache (for network filesystems).
FS_ODD_RENAME	"Rename" operations are "move" operations (for network filesystems).

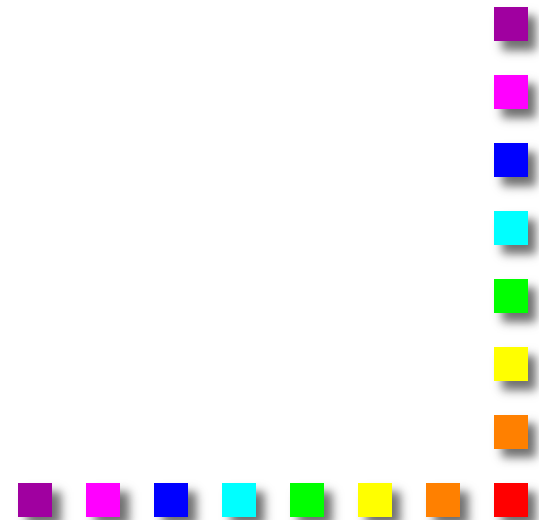


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- **Filesystem Handling**
- Pathname Lookup
- System Call Implementation
- File Locking



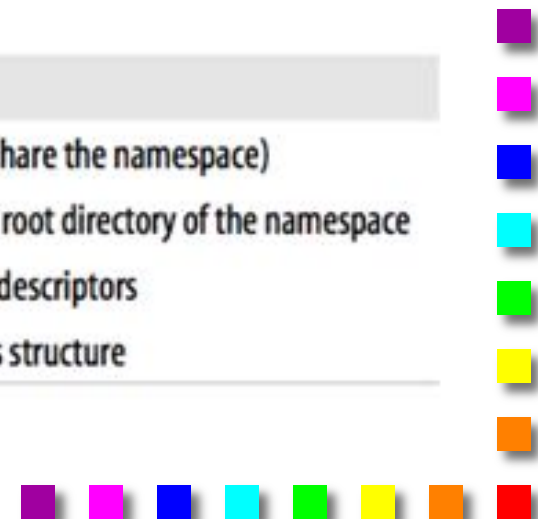
上海交通大学



Filesystem Handling

- Root filesystem
- Mount point
 - /proc virtual filesystem is a child of the system's root filesystem
- Namespaces: in Linux 2.6, each process might have its own tree of mounted filesystems
 - Namespace structure

Type	Field	Description
atomic_t	count	Usage counter (how many processes share the namespace)
struct vfsmount *	root	Mounted filesystem descriptor for the root directory of the namespace
struct list_head	list	Head of list of all mounted filesystem descriptors
struct rw_semaphore	sem	Read/write semaphore protecting this structure



Mount a Filesystem (1)

- Filesystem mounting

- *mount -t ext2 /dev/fd0 /flp*

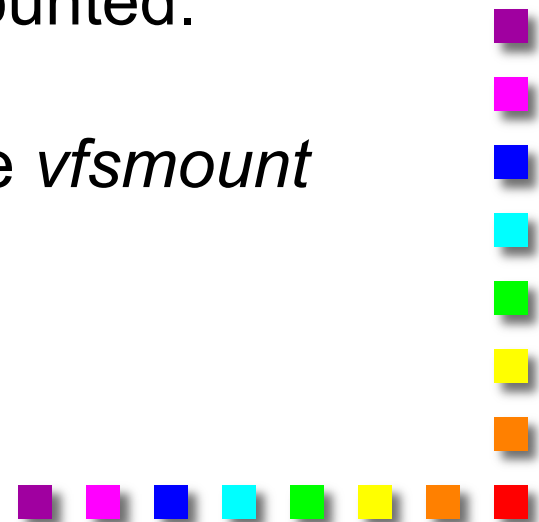
- It's possible in Linux to mount the same filesystem several times

- There is **only one superblock object**, no matter how many times a filesystem has been mounted.

- Mounted filesystem descriptor: of type *vfsmount* (Tabel 12-12)



上海交通大学



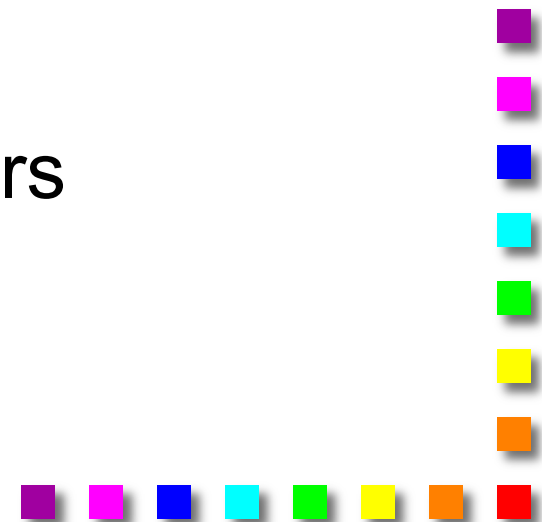
Mount a Filesystem (2)

- Filesystem mounting
 - ...
 - Looks up the pathname of the mount point by invoking `path_lookup()`
 - Examines the mount flags to determine what has to be done
 - `do_kern_mount()`
 - Terminate the pathname lookup of the mount point
 - Return



Unmount a Filesystem

- Filesystem unmounting
 - Invoke `path_lookup()` to look up the mount point pathname
 - Check correctness and privileges
 - Invoke `do_unmount()`
 - Decrease corresponding counters
 - Return

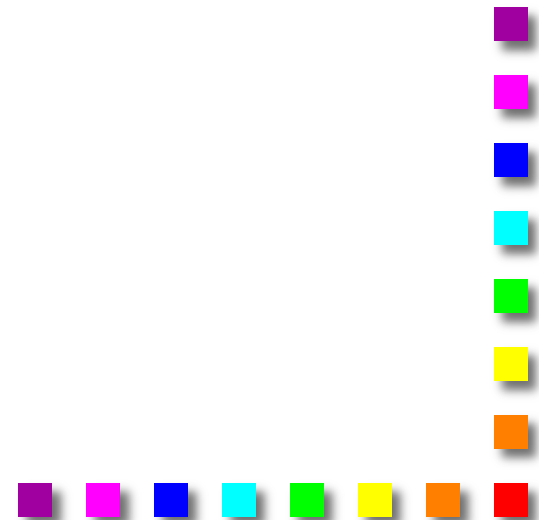


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking

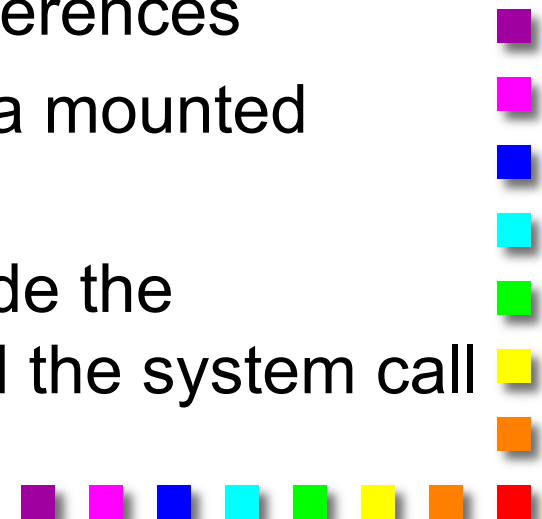


上海交通大学



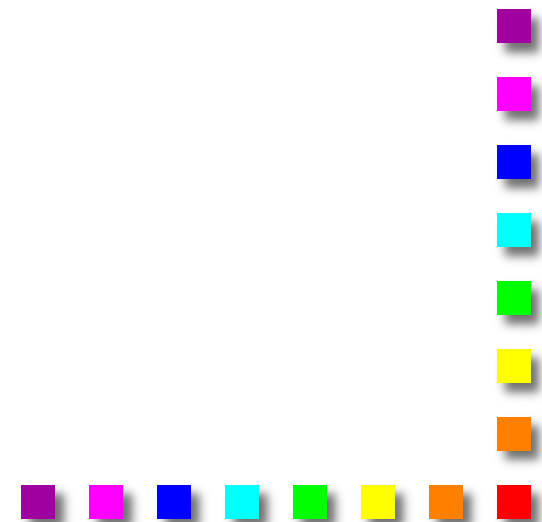
Pathname lookup (1)

- Pathname -> inode
- Analyzing the pathname and breaking it into a sequence of filenames
- Hard points
 - Check Access Rights of each directory
 - Symbolic Link
 - Symbolic links may induce circular references
 - A filename can be the mount point of a mounted filesystem
 - Pathname lookup has to be done inside the namespace of the process that issued the system call



Pathname lookup (2)

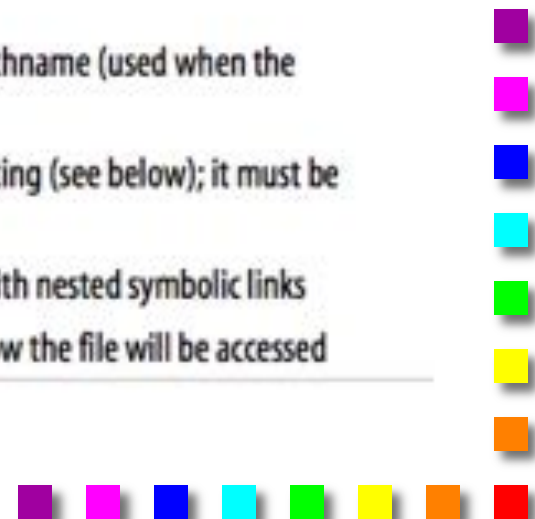
- Analyzing the pathname and breaking it into a sequence of filenames
 - Absolute pathname, relative pathname
 - Current->fs->root, current->fs->pwd
 - Path_lookup(): return the nameidata structure
 - Standard pathname lookup
 - Parent pathname lookup
 - Lookup of symbolic links



nameidata struct

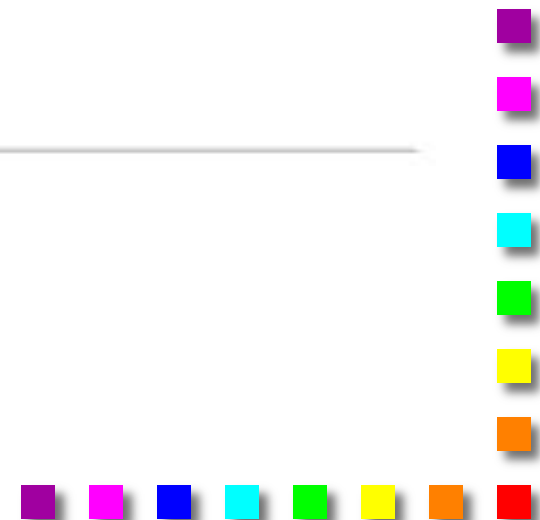
- The **dentry** and **mnt** fields point respectively to the dentry object and the mounted filesystem object of the **last resolved component** in the pathname

Type	Field	Description
struct dentry *	dentry	Address of the dentry object
struct vfs_mount *	mnt	Address of the mounted filesystem object
struct qstr	last	Last component of the pathname (used when the LOOKUP_PARENT flag is set)
unsigned int	flags	Lookup flags
int	last_type	Type of last component of the pathname (used when the LOOKUP_PARENT flag is set)
unsigned int	depth	Current level of symbolic link nesting (see below); it must be smaller than 6
char[] *	saved_names	Array of pathnames associated with nested symbolic links
union	intent	One-member union specifying how the file will be accessed



Flags of the lookup operation

Macro	Description
LOOKUP_FOLLOW	If the last component is a symbolic link, interpret (follow) it
LOOKUP_DIRECTORY	The last component must be a directory
LOOKUP_CONTINUE	There are still filenames to be examined in the pathname
LOOKUP_PARENT	Look up the directory that includes the last component of the pathname
LOOKUP_NOALT	Do not consider the emulated root directory (useless in the 80x86 architecture)
LOOKUP_OPEN	Intent is to open a file
LOOKUP_CREATE	Intent is to create a file (if it doesn't exist)
LOOKUP_ACCESS	Intent is to check user's permission for a file



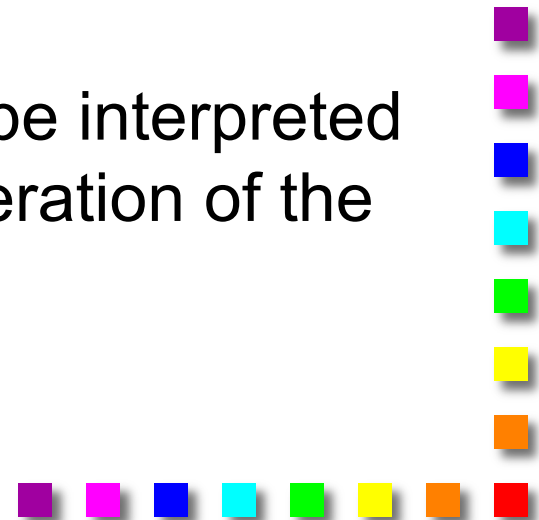
Steps

- Initialize **nd** (type: **nameidata**)
- Acquires for reading the current->fs->lock read/write semaphore of the current process.
- Obtain addresses of mounted filesystem object, and dentry object.
- Store addresses of the above two objects in *nd*.
- Releases the current->fs->lock read/write semaphore of the current process
- Sets the total_link_count field in the descriptor of the current process to 0
- Invokes the link_path_walk() function



Parent Pathname Lookup

- Pathname lookup
 - The real target of a lookup operation is not the last component of the pathname, but the next-to-last one
 - The LOOKUP_PARENT flag is set
 - The last component is not interpreted at all
- Lookup of Symbolic Links
 - If the inode is a symbolic link, it must be interpreted before proceeding with the lookup operation of the original pathname.

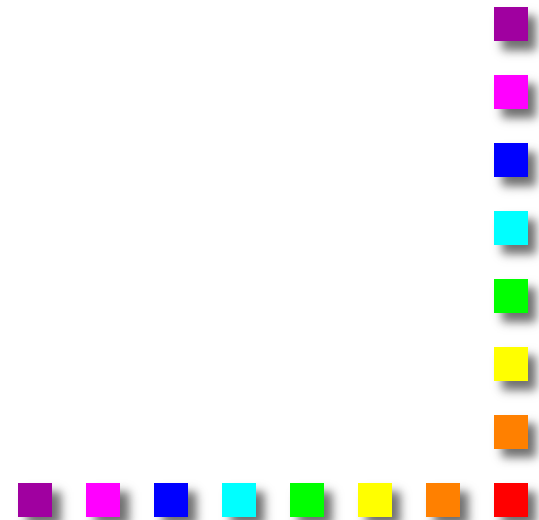


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking

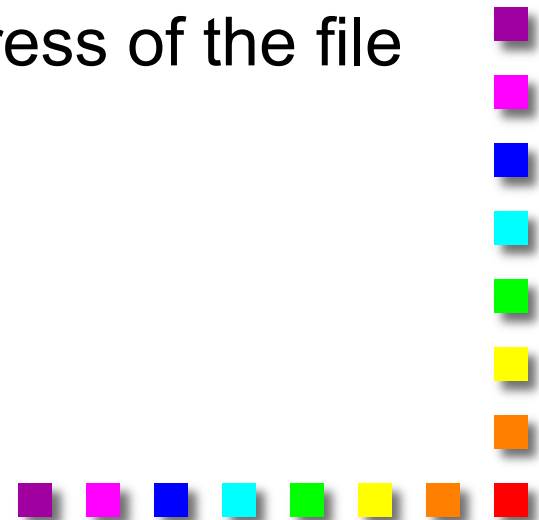


上海交通大学



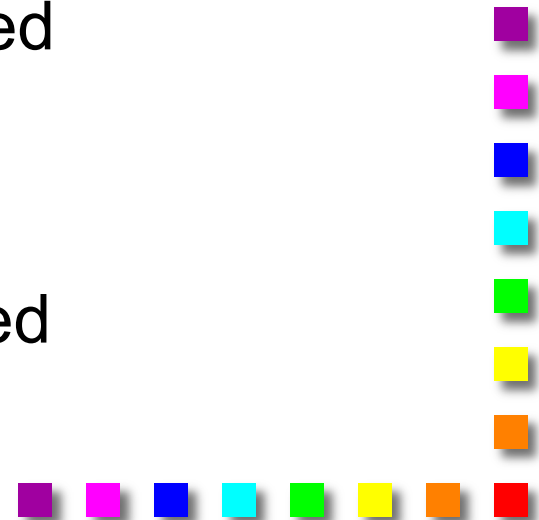
Open()

- Open(): filename, flags, mode, Return file descriptor
 - 1. Invokes getname() to read the file pathname from the process address space
 - Invokes get_unused_fd() to find an empty slot in current->files->fd.
 - Invokes the filp_open() function
 - Sets current->files->fd[fd] to the address of the file object returned
 - Returns fd



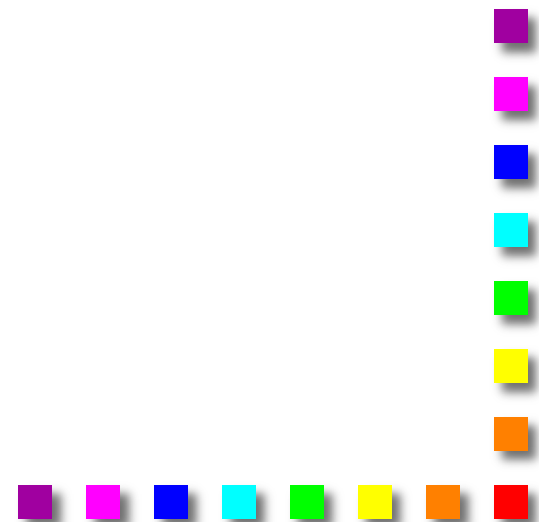
Read(), Write()

- Read(), Write(): file descriptor, addr of a buffer, count
 - Derive from *fd* the address file of the corresponding file object
 - Check flags
 - Perform a coarse check on the parameters
 - Check whether there are conflicting mandatory locks for the file portion to be accessed
 - Transfer data
 - Release file object
 - Return the number of bytes transferred



Close()

- Close(): File descriptor *fd*
 - Gets the to-be-closed file object address stored in `current->files->fd[fd]`
 - Sets `current->files->fd[fd]` to NULL
 - Invokes `filp_close( )`
 - Returns 0 or an error code

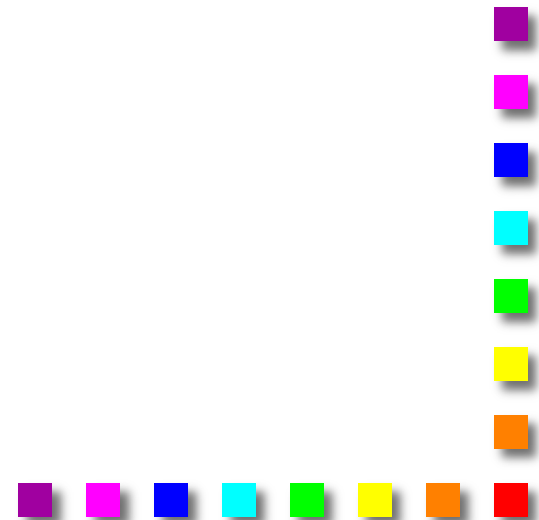


Outline

- Role of VFS
- VFS Data Structures
- Filesystem Types
- Filesystem Handling
- Pathname Lookup
- System Call Implementation
- File Locking



上海交通大学



File Locking

- Advisory locks: by POSIX
 - Based on `fcntl()` system call
 - Possible to lock an arbitrary region of a file
- Mandatory locks: by System V Release 3
 - The kernel checks every invocation of `open()`, `read()`, `write()` system calls does not violate a mandatory lock
- Linux supports both + `fcntl()`, `flock()` and `lockf()` system calls



File Locking (2)

■ File_lock struct

Type	Field	Description
struct file_lock *	fl_next	Next element in list of locks associated with the inode
struct list_head	fl_link	Pointers for active or blocked list
struct list_head	fl_block	Pointers for the lock's waiters list
struct files_struct *	fl_owner	Owner's files_struct
unsigned int	fl_pid	PID of the process owner
wait_queue_head_t	fl_wait	Wait queue of blocked processes
struct file *	fl_file	Pointer to file object
unsigned char	fl_flags	Lock flags
unsigned char	fl_type	Lock type
loff_t	fl_start	Starting offset of locked region
loff_t	fl_end	Ending offset of locked region
struct fasync_struct *	fl_fasync	Used for lease break notifications
unsigned long	fl_break_time	Remaining time before end of lease
struct file_lock_operations *	fl_ops	Pointer to file lock operations
struct lock_manager_operations *	fl_mops	Pointer to lock manager operations
union	fl_u	Filesystem-specific information



■ The end



上海交通大学

